Дизајн и архитектура на софтвер

Домашна 4

Во рамки на оваа домашна работа беше извршено рефакторирање на апликацијата развиена во третата домашна работа која претставува систем за анализа на криптовалути и управување со корисници.

Целта на рефакторирањето е подобрување на структурата на кодот, јасноста, модуларноста и одржливоста, како и подготовка на апликацијата за работа со микросервиси и контејнери. За таа цел, апликацијата е реорганизирана според **MVC (Model–View–Controller)** архитектонскиот шаблон, со јасно одделување на одговорностите помеѓу различните слоеви.

1. MVC (Model–View–Controller)

MVC (Model–View–Controller) е архитектонски шаблон кој ја дели апликацијата на три главни компоненти:

- **Model** – ги претставува податоците и бизнис ентитетите.
- **View** – одговорен за прикажување на податоците (кориснички интерфејс или API одговори).
- **Controller** – ги обработува корисничките барања и ја поврзува View со Model.

Овој пристап овозможува подобра организација на кодот и полесно одржување на апликацијата.

Причини за избор на MVC шаблонот

MVC шаблонот беше избран како архитектонско решение поради неговата способност да овозможи јасна и логична поделба на одговорностите во рамките на апликацијата. Со раздвојување на бизнис логиката, обработката на корисничките барања и приказот на податоците, кодот станува поорганизиран и полесен за разбирање, што значително ја подобрува неговата читливост.

Дополнително, примената на MVC архитектурата овозможува полесно проширување и надградување на апликацијата во иднина. Промените во еден дел од системот, како што се бизнис логиката или начинот на прикажување на податоците, може да се направат без значително влијание врз останатите делови од апликацијата. Ова ја зголемува флексибилноста и одржливоста на кодот.

MVC шаблонот исто така овозможува полесно и поефикасно тестирање на поединечни компоненти. Со јасно дефинирани слоеви, секој дел од апликацијата може да се тестира независно, што ја зголемува сигурноста и стабилноста на системот.

Покрај тоа, MVC претставува стандардна и широко прифатена архитектура кај Java и Spring Boot апликациите. Неговата употреба се смета за добра пракса во современиот софтверски развој, што дополнително ја оправдува неговата примена во оваа апликација.

1.1 Model слој

Model слојот претставува основа на апликацијата и е задолжен за дефинирање на податоците и бизнис ентитетите со кои системот работи. Во рамките на MVC архитектурата, овој слој ги содржи класите кои ја опишуваат структурата на податоците, нивните атрибути и меѓусебни релации, без да содржат логика поврзана со кориснички интерфејс или обработка на HTTP барања.

Во апликацијата, Model слојот е имплементиран преку повеќе класи кои покриваат различни аспекти на системот. Моделот **User** ги претставува регистрираните корисници и содржи информации потребни за автентикација и авторизација. За потребите на процесот на логирање и регистрација се користат и посебни модели, како **LoginRequest**, **RegisterRequest** и **JwtResponse**, кои овозможуваат јасна и структурирана размена на податоци помеѓу клиентот и серверот.

Покрај корисничкиот дел, значаен сегмент од Model слојот е посветен на анализа на криптовалути. Моделот **Cryptocurrency** ги претставува криптовалутите кои се следат во системот, додека моделот **DailyData** содржи дневни податоци поврзани со цената за достапните криптовалути.

1.2 View слој

Во рамките на оваа апликација, која е имплементирана како REST API, View слојот не е претставен преку класичен графички кориснички интерфејс. Наместо тоа, улогата на View слојот ја имаат HTTP одговорите кои се враќаат од Controller слојот кон клиентот, најчесто во JSON формат.

View слојот е одговорен за прикажување на резултатите од обработката на барањата, односно за структурирано претставување на податоците што ги добива клиентот. За таа цел, во апликацијата се користат посебни response модели, како што е JwtResponse, како и

ResponseEntity објекти кои овозможуваат враќање на податоци заедно со соодветни HTTP статус кодови.

Со ваков пристап се овозможува јасна комуникација помеѓу серверот и клиентските апликации, додека визуелниот приказ и корисничкиот интерфејс се оставени да бидат имплементирани на клиентска страна.

1.3 Controller слој

Controller слојот претставува посредник помеѓу клиентот и сервисниот слој во апликацијата. Неговата главна улога е да прима HTTP барања, да ги проследува кон сервисите, и да враќа соодветни одговори кон клиентската апликација, обично во JSON формат. Контролерите не содржат бизнис логика, туку обезбедуваат чист интерфејс за комуникација и обработка на барањата.

Во апликацијата, Controller слојот е имплементиран преку повеќе REST контролери, кои овозможуваат пристап до различни функционалности преку дефинирани API endpoints. Сите контролери се означени со @RestController, со што се овозможува автоматско враќање на податоците во JSON формат, како и соодветна комуникација со клиентските апликации.

Во нашата апликација се имплементирани следниве контролери:

- **AnalysisController**

  AnalysisController е задолжен за операции поврзани со техничка анализа на криптовалути.
  Тој комуницира со надворешен микросервис преку RestTemplate и овозможува следниве функционалности:

  - **getTechnicalAnalysis** – добива техничка анализа за одредена криптовалута и период.
  - **getLstmAnalysis** – овозможува LSTM анализа за предвидување на трендови.
  - **getOnchainSentiment** – комбинирана анализа на on-chain податоци и sentiment индикатори.

  Контролерот ги проследува барањата кон надворешниот микросервис и ги враќа резултатите како JSON. Во случај на грешка, се враќа HTTP 500 статус, што обезбедува контролиран и предвидлив одговор кон клиентот.

- **AuthController**

AuthController е одговорен за функционалностите поврзани со автентикација и авторизација на корисниците. Преку овој контролер се овозможуваат операции за регистрација и логирање на корисници. При регистрација, контролерот ги прима податоците од клиентот и ги проследува кон сервисниот слој, каде што се извршува валидација и зачувување на корисникот. При логирање се користи Spring Security за автентикација на корисникот, по што се генерира JWT токен кој се враќа како одговор до клиентот.

- **CryptocurrencyController**

CryptocurrencyController е одговорен за управување со криптовалутите.

Главните операции вклучуваат:

- **getAllCryptocurrency** – враќа список со сите криптовалути.
- **getCryptocurrencyById** – враќа информација за конкретна криптовалута, идентификувана преку ID.
- **search** – овозможува пребарување на криптовалути според име

Овој контролер ги повикува методите на CryptocurrencyService за добивање на податоците и ги враќа како JSON одговори заедно со соодветниот HTTP статус. Ако не се најде криптовалута, се фрла CryptocurrencyNotFoundException, која се обработува во сервисниот слој.

- **DailyDataController**

DailyDataController се грижи за дневните податоци на криптовалутите.
Неговите функции се:

- **getAllPaged** – враќа податоци на дневно ниво за сите криптовалути.
- **getDailyDataById** – враќа специфични дневни податоци идентификувани со ID.
- **getDailyForCrypto** – добива листа на дневни податоци за конкретна криптовалута.

Контролерот ги повикува методите на DailyDataService за добивање на потребните податоци и враќа ResponseEntity со соодветен HTTP статус. Ако не се најдат податоци, се фрла DailyMarketDataNotFoundException.